# GRAPH BASED SSL

## Label Propagation Algorithm (LPA)

"propagates" estimated label at each node to neighbours based on weight

$\Rightarrow$ new label = weighted sum of labels of neighbours.

### Steps

① Construct adjacency matrix A based on connections in graph

② Compute transition matrix T :

(a) Row normalise A

(b) $\overbrace{T_{ij} = \frac{w_{ij}}{\sum_{N(v_i)} w_{ik}}}$ where $w_{ij} = \begin{cases} 1 \text{ if } v_i \in N & [\text{K-NN}] \\ e^{\frac{-\|v_i - v_j\|^2}{2\sigma^2}} & [\text{Gaussian}] \end{cases}$

→ just row normalisation

→ W forms affinity matrix.

③ Initialise $Y^{(0)}$ : rows = nodes

cols = P(class$_i$)

→ one hot encode to init

④ Until convergence :

(a) Propagate

$Y_{prop}^{(t)} = T Y^{(t-1)}$

(b) Row normalise

(c) Clamp labelled nodes back to og probability (we know these are true)

$Y^{(t)} = \text{clamp}(Y_{prop}^{(t)})$

⑤ Choose final label using argmax

## Label Spreading Algorithm (LSA)

soft clamping ⟶ labelled nodes do not stay fixed.

① Affinity matrix W, init $Y^{(0)}$

$w_{ij} = e^{\frac{-\|x_i - x_j\|^2}{2\sigma}}$ . $w_{ii} = 0$

② Symmetric normalised matrix:

$$\boxed{S = D^{-\frac{1}{2}} W D^{-\frac{1}{2}}}$$

Symmetric normalized Laplacian

diagonal matrix

$D_{ii}$ = sum of $i^{th}$ row of W

$D^{-1}A$ → row normalised

$AD^{-1}$ → col normalised

③ Until convergence :

Spread

$$\boxed{F^{(t)} = \underbrace{\alpha S F^{(t-1)}} + \underbrace{(1-\alpha) Y^{(0)}}}$$

like $TY^{(t-1)}$, but with α.

information from original labels

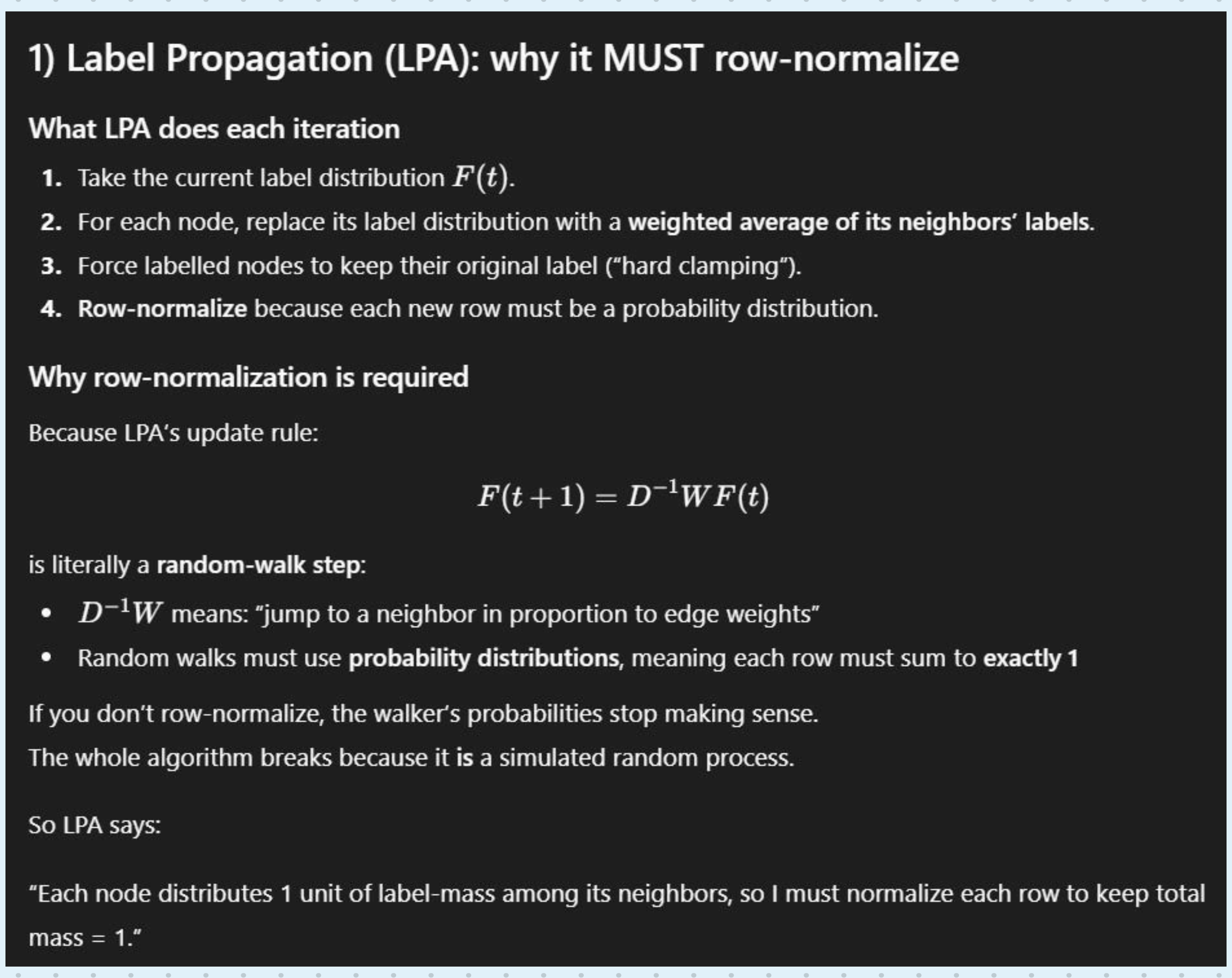
**Hyperparameter: α** (0<α<1)

Controls the extent of clamping

α → proportion of info from neighbours

1-α → proportion from original labels

④ Argmax for labels



### 1) Label Propagation (LPA): why it MUST row-normalize

**What LPA does each iteration**

1. Take the current label distribution $F(t)$.
2. For each node, replace its label distribution with a **weighted average of its neighbors' labels.**
3. Force labelled nodes to keep their original label ("hard clamping").
4. **Row-normalize** because each new row must be a probability distribution.

**Why row-normalization is required**

Because LPA's update rule:

$$F(t+1) = D^{-1}WF(t)$$

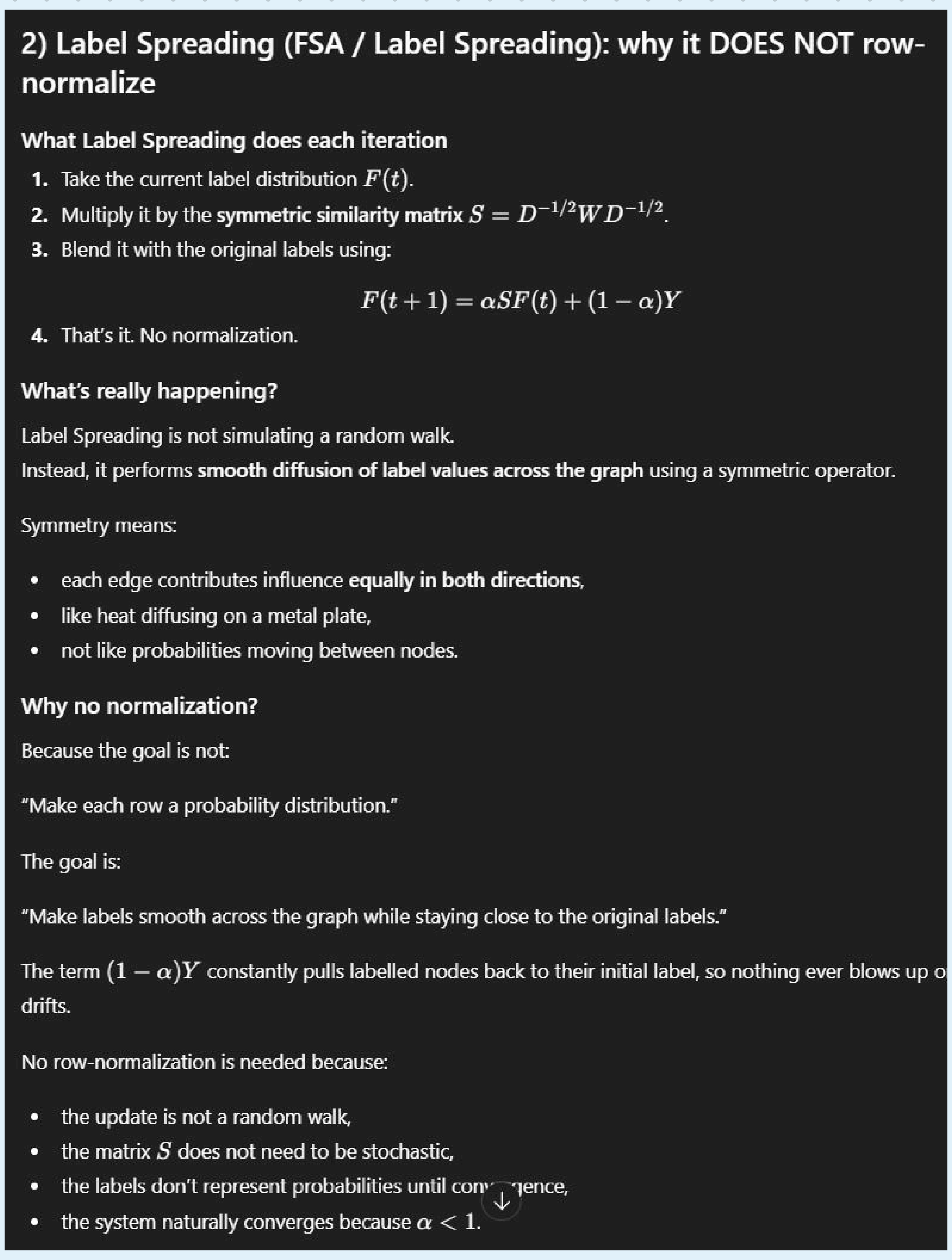
is literally a **random-walk step**:

- $D^{-1}W$ means: "jump to a neighbor in proportion to edge weights"
- Random walks must use **probability distributions**, meaning each row must sum to **exactly 1**

If you don't row-normalize, the walker's probabilities stop making sense.
The whole algorithm breaks because it **is** a simulated random process.

So LPA says:

"Each node distributes 1 unit of label-mass among its neighbors, so I must normalize each row to keep total mass = 1."



### 2) Label Spreading (FSA / Label Spreading): why it DOES NOT row-normalize

**What Label Spreading does each iteration**

1. Take the current label distribution $F(t)$.
2. Multiply it by the **symmetric similarity matrix** $S = D^{-1/2}WD^{-1/2}$.
3. Blend it with the original labels using:

$$F(t+1) = \alpha SF(t) + (1-\alpha)Y$$

4. That's it. No normalization.

**What's really happening?**

Label Spreading is not simulating a random walk.
Instead, it performs **smooth diffusion of label values across the graph** using a symmetric operator.

Symmetry means:

- each edge contributes influence **equally in both directions**,
- like heat diffusing on a metal plate,
- not like probabilities moving between nodes.

**Why no normalization?**

Because the goal is not:

"Make each row a probability distribution."

The goal is:

"Make labels smooth across the graph while staying close to the original labels."

The term $(1-\alpha)Y$ constantly pulls labelled nodes back to their initial label, so nothing ever blows up o drifts.

No row-normalization is needed because:

- the update is not a random walk,
- the matrix $S$ does not need to be stochastic,
- the labels don't represent probabilities until convergence,
- the system naturally converges because $\alpha < 1$.